

ЛАБОРАТОРНА РОБОТА №4

ОСНОВИ РОБОТИ З БІБЛІОТЕКОЮ REACT

Мета роботи: отримання базових навичок роботи з бібліотекою React, призначеної для побудови складних інтерфейсів, що масштабуються та реконфігуруються.

Хід роботи:

1. Додатки:

Додаток А: код.

2. Контрольні питання:

1) Призначення бібліотеки React:

Реакт це бібліотека для створення UI, його філософія - $UI = f(state)$. Також він має свої особливості: декларативність(розробник описує вигляд інтерфейсу для певного стану, а реакт самдбає яоновлювати ДОМ дерево).

2) Життєвий цикл компонентів React:

1. `componentDidMount`: дії які виконується коли комопонет першийраз відрендерився (зазвичай виконуються у хуці `useEffect`) 2. `componentDidUpdate`: виконується коли оновлення вжезастосовано у DOM.

3. `componentWillUnmount`: Виконується перед видаленнямкомпоненту, зазвичай на цьому етапі відписуються від подій, закривають таймери та скасовуються підписки.

3) Сформулювати визначення стану компонента React: Стан - це вбудований об'єкт, який містить дані про поточнуситуацію у середені компонента .

Висновок: Під час виконання лабораторної роботи я отримав базові навички роботи з бібліотекою React, призначеної для побудовискладних інтерфейсів, що масштабуються та реконфігуруються.

Додаток А

```
"use client"
```

```

import { Todo, TodoCard, useTodo } from "@src/entity/todo"
import {
  Button,
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerFooter,
  DrawerHeader,
  DrawerTitle,
  DrawerTrigger,
  Spinner,
} from "@src/shared/ui"
import { cn } from "@src/shared/utils"
import { Check, Inbox, Trash2 } from "lucide-react"
import { useState } from "react"
const TodoListItem = ({ todo }: { todo: Todo }) => {
  const [isOpen, setIsOpen] = useState(false)
  const {
    actions: { deleteTodo, toggleTodo },
  } = useTodo()
  const formattedDate = new Intl.DateTimeFormat("uk-UA", {
    day: "numeric",
    month: "long",
    year: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(todo.createdAt))
  return (
    <li>
      <Drawer open={isOpen} onOpenChange={setIsOpen}>
        <DrawerTrigger asChild>
          <div className="cursor-pointer transition-opacity hover:opacity-80">
            <TodoCard todo={todo} />
          </div>
        </DrawerTrigger>
        <DrawerContent className="mx-auto max-w-2xl px-2 pb-4 sm:px-4">
          <DrawerHeader className="text-left">
            <DrawerTitle className="text-2xl">{todo.title}</DrawerTitle>
            <DrawerDescription className="mt-4 text-base break-words whitespace-pre-wrap text-foreground">
              {todo.description || (
                <span className="text-muted-foreground"> Опис відсутній</span>
              )}
            </DrawerDescription>
          </DrawerHeader>

```

```

        <div className="px-4 py-4 text-sm text-muted-foreground">
          <p>Створено: {formattedDate}</p>
          <p>Статус: {todo.isCompleted ? "Виконано" : "Не виконано"}</p>
        </div>
        <DrawerFooter className="mt-4 flex flex-col gap-2 sm:flex-row sm:justify-end">
          <Button
            variant="destructive"
            onClick={() => {
              deleteTodo(todo.id)
              setIsOpen(false)
            }}
            className="sm:w-auto"
          >
            <Trash2 className="mr-2 h-4 w-4" />
            Видалити
          </Button>
          <Button
            variant={todo.isCompleted ? "secondary" : "success"}
            onClick={() => {
              toggleTodo(todo.id)
              setIsOpen(false)
            }}
            className="sm:w-auto"
          >
            <Check className="mr-2 h-4 w-4" />
            {todo.isCompleted ? "Повернути в роботу" : "Виконати"}
          </Button>
        </DrawerFooter>
      </DrawerContent>
    </Drawer>
  </li>
)
}
export const TodoList = () => {
  const {
    values: { todos, isLoading },
  } = useTodo()
  if (isLoading) {
    return (
      <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2
transform">
        <Spinner />
      </div>
    )
  }
  if (!todos.length) {

```

```
    return (  
      <div className="absolute top-1/2 left-1/2 flex -translate-x-1/2 -translate-y-1/2  
transform flex-col items-center justify-center gap-2 text-muted-foreground">  
        <Inbox size={52} />  
        <p>Ще немає задач</p>  
      </div>  
    )  
  }  
  return (  
    <ul  
      className={cn(  
        "mt-4 flex flex-wrap items-stretch justify-center gap-4",  
        todos.length < 8 && "justify-normal"  
      )}  
    >  
      {todos.map((todo) => (  
        <TodoListItem key={todo.id} todo={todo} />  
      ))}  
    </ul>  
  )  
}
```

